Claude Code CLI 源码分析：从 Prompt 架构到工业级 AI Agent缓存黑科技+自我进化+多Agent协作+遥测监控

huangserva
@servasyy_ai · 3h

Follow

7 29 102 21K

最近拿到了 Claude Code CLI 的源码。

之前写过 OpenClaw 的 9 层 Prompt 架构，当时就在想：Anthropic 这种量级的公司，他们会怎么设计 AI Agent?

看完源码才发现，这不是“Prompt 工程”那么简单，而是**缓存架构 + 自我进化记忆 + 多模式编排**的完整基础设施。

constants/prompts.ts 展示的是两层缓存架构，cache_edits 实现的是增量删除机制，autoDream 构建的是自我进化的记忆系统，PROACTIVE 模式切换的是主动执行的编排逻辑。

这才是为什么 Claude Code 能支持无限长对话，为什么删除几百条消息后响应速度还是那么快，为什么 Agent 能从“记录对话”进化到“学习知识”。

这不是 prompt 写得好不好的问题，是整个系统架构为长对话、自主工作、持续进化优化到了极致。

一、System Prompt 的两层缓存架构：静态 vs 动态

大部分 AI 应用的 prompt 是一整块发给模型的。每次对话都要重发，浪费 token 也浪费钱。

Claude Code 把 prompt 切成两层：

核心设计

1. 静态前缀用 scope: 'global'

Anthropic 后端跨用户、跨组织共享缓存。所有人的所有会话都复用同一份。这部分内容几乎不变：

- 身份定义（Intro）
- 系统规则（System）
- 任务规范（Doing Tasks）
- 操作安全（Actions）
- 工具使用指南（Using Tools）
- 风格要求（Tone & Style）
- 效率要求（Output Efficiency）

2. 动态部分用 scope: null

每个会话独立，只在 /clear 或 /compact 时失效。包含会话特定的：

- 会话指引（Session Guidance）

- 记忆（Memory）
- 环境信息（Env Info）
- MCP 指令（MCP Instructions）
- 语言偏好（Language）
- 输出风格（Output Style）
- 草稿本（Scratchpad）
- Token 预算（Token Budget）

3. 分界线（BOUNDARY MARKER）

这是整个设计的关键。分界线确保动态部分变化时，静态部分的缓存不会失效。两者完全解耦。

为什么这样设计？

- 静态部分大约 5-8K token，但可以跨所有用户共享
- 动态部分每次会话不同，通常也是 5-10K token
- 分界线让两者解耦，动态变化不影响静态缓存
- 节省的不是绝对数量，而是**跨数百万用户共享这个乘数效应**

缓存管理机制

动态部分不是每次重算，而是用 systemPromptSection() 注册 + memoize，只在 /clear 或 /compact 时清缓存。

例外：MCP Instructions 使用了 DANGEROUS_uncachedSystemPromptSection，因为 MCP 服务器会中途连接/断开，需要实时更新。

效果

- Claude Code 启动几乎是瞬间的
- 新会话只需要发送动态部分
- 长对话的成本显著降低（短对话节省不明显）

二、四层递进的 Compact 架构：从轻量清理到全量压缩

Claude Code 的压缩不是单一机制，而是**四层递进的压缩策略**，根据上下文压力逐级升级。

整体架构

```
上下文接近上限
 ↓
Layer 1: MicroCompact（轻量清理）
 ├── 缓存已冷 → 直接清空旧 tool_result
 └── 缓存还热 → cache_edits 增量删除
 ↓ 不够?
Layer 2: SessionMemoryCompact（会话记忆压缩）
 ├── 保留最近 10K-40K token 的消息
 └── 用 Session Memory 替代旧对话
 ↓ 还不够?
Layer 3: Full Compact（全量压缩）
 ├── Fork 一个 agent 做摘要
 ├── 复用主对话的 prompt cache
 └── 图片替换为 [image] 标记
 ↓ 还不够?
Layer 4: PTL Retry（终极兜底）
 └── 从头部砍掉最旧的消息组，最多砍 20%
```

Layer 1: MicroCompact + Cache Edits（黑科技）

这是 Anthropic API 的未公开功能，核心是**缓存不失效的增量删除**。

问题场景

长对话中，旧的 tool_result（工具调用结果）会占满上下文。一个典型的长对话可能有 100K+ token 的 tool_result。

传统做法有两种：

1. **直接删掉旧消息** → 整个 prompt cache 失效 → 下一轮全部重发 → 费用飙升（$1+ 一轮）、延迟明显（5-10 秒）
2. **不删除** → 上下文溢出，对话中断

为什么删除旧消息会导致缓存失效?

这是理解 Cache Edits 价值的关键。

Prompt Cache 的工作原理：前缀完全匹配

假设你的消息序列是这样的：

[System Prompt] + [消息1] + [消息2] + [消息3] + [消息4] + [消息5]

Claude 会把这整个序列缓存起来。下一轮对话，你发送：

[System Prompt] + [消息1] + [消息2] + [消息3] + [消息4] + [消息5] + [消息6]

Claude 发现前面的部分和缓存完全一致，直接复用缓存，只处理新的 【消息6】。

但如果你删掉了消息 2：

[System Prompt] + [消息1] + [消息3] + [消息4] + [消息5] + [消息6]

Claude 发现从消息 1 之后，序列就对不上了：

- 缓存里是：【消息 1】 + 【消息 2】 + 【消息 3】.....
- 你发的是：【消息 1】 + 【消息 3】 + 【消息 4】.....

结果：缓存失效，整个序列需要重新处理。

这就像从书架上抽掉一本书，书的顺序变了，别人来找书时发现对不上，要重新整理书架。

Cache Edits 的解法

不修改本地消息，在 API 层发送删除指令：

```
{
  "type": "cache_edits",
  "edits": [
    { "type": "delete", "cache_reference": "tool_use_id_A" },
    { "type": "delete", "cache_reference": "tool_use_id_B" }
  ]
}
```

工作原理（3 步）

1. 给每个块贴上 ID

每个 tool_result 发送时带上 cache_reference: tool_use_id，就像给缓存中的每个块贴上条形码。

目的：方便后续通过 ID 引用这些块。

2. 告诉服务端“跳过这些块”

通过 cache_edits 发送删除指令，告诉服务端“把这些块标记为不可见”。

关键点：**数据还在缓存里，只是被标记为“跳过”**。本地消息也不动，只是在 API 层告诉服务端“别把这些块喂给模型”。

3. 保存标记指令，供后续请求使用

因为缓存里的原始数据还在（只是被标记为跳过），所以每一轮都需要重新发送 cache_edits。

Claude Code 会把这个标记指令保存下来，自动在后续请求中附带，确保那些块持续被跳过。

效果

- **缓存前缀没变** → 下一轮还是命中
- **模型看到的上下文变短了** → 省 token
- **原始数据还在服务端** → 只是被标记为“跳过”

什么时候会用 Cache Edits?

Claude Code 有一个自动压缩机制叫 microCompactMessages()，当 tool_result 太多时，会自动清理旧的。

但清理方式有两种，取决于**缓存是否还热着**：

场景 1：缓存已经过期了（超过 5 分钟）

直接删除旧 tool_result 的内容，反正缓存已经失效了，删了也不影响。

场景 2：缓存还热着（5 分钟内）

用 Cache Edits 增量删除，保持缓存命中。

具体流程：

```
microCompactMessages()
  |
  ├── 时间触发 → 缓存已冷，直接清空旧 tool_result 内容
  |
  └── Cached MC → cache_edits 路径
      ├── 注册 tool_result → track 哪些可压缩
      ├── 计算哪些该删（保留最近 N 个）
      ├── 生成 cache_edits 块
      ├── 保存 edits 用于后续轮次
      └── 本地消息不变，API 层处理
```

关键点：Cache Edits 只在缓存还热的时候用，这样才能保持缓存命中。

对比：传统方法 vs Cache Edits

cache_edits 的优势在于**只删 tool_result 不动其他部分**，而传统 compact 虽然 system prompt 部分仍能命中缓存，但 tool_result 部分会失效，需要重发。

Layer 2: SessionMemoryCompact — 外科手术式的精确切割

当 MicroCompact 清理完 tool_result 还不够时，就要动真格的了：切掉旧对话。

但这不是简单的“砍前半段”。Claude Code 的做法是**外科手术式的精确切割**，确保不会把一个完整的交互拆散。

核心规则：

- 保留最近 10K-40K token 的消息（根据上下文压力动态调整）
- 用一条 Session Memory 替代旧对话（“之前我们讨论了.....”）
- **绝不拆散 tool_use 和 **tool_result — 工具调用和结果必须成对出现
- **绝不拆散** thinking **块和同** message.id **的** tool_use — 思考过程和执行动作必须在一起

为什么这样设计?

如果你把 tool_use 留下但删掉 tool_result，模型会以为工具调用还没执行完，陷入等待状态。如果你把 thinking 块删掉但留下 tool_use，模型会不知道为什么要调用这个工具。

所以 Claude Code 的切割逻辑是：**从保留区的起点向后扩展，直到满足最小 token 数和最小消息数，并且不破坏任何完整的交互组。**

这是一个“宁可多留一点，也不能拆散”的保守策略，确保上下文的语义完整性。

Layer 3: Full Compact — 复用缓存的摘要魔法

当前两层都不够时，就要祭出大招了：Fork 一个 agent 做全量摘要。

但这里有个精妙的设计：**摘要 agent 的 system prompt + tools + model 和主对话完全一致。**

为什么？因为这样可以**直接命中主对话的 prompt cache**，大幅降低 token 成本。

工作流程：

```
主对话上下文快满了
 ↓
Fork 一个摘要 agent（复用主对话的 system prompt）
 ↓
图片清理 — 把所有 image/document 块替换为 [image]/[document] 标记
  （图片占大量 token 但对摘要无用）
 ↓
摘要 agent 读取对话历史，生成摘要
  （因为 system prompt 一致，直接命中缓存）
 ↓
摘要完成后，自动注入关键上下文：
 - 最近 5 个文件的最新内容（每文件 ≤5K token，总预算 50K）
```

- 已激活的 Skills（每个 ≤5K token，总预算 25K）
- MCP Instructions Delta
↓
主对话继续，上下文变短了，但关键信息没丢

为什么要自动注入文件和 Skills?

因为压缩后，模型会忘记之前读过哪些文件、激活过哪些技能。如果不自动恢复，下一轮对话模型会说“我需要先读取 xxx 文件”，浪费一轮交互。

自动注入的逻辑是：**把最近操作过的文件和技能重新喂给模型，但控制总量（文件 50K，Skills 25K），不会让上下文再次爆掉。**

这是一个“压缩后恢复”的闭环设计，确保模型不会因为压缩而丢失工作记忆。

Layer 4: PTL Retry — 最后的保险机制

即使是摘要请求本身，也可能触发 prompt_too_long。

这种情况通常发生在极端场景：对话历史太长，连摘要 agent 都塞不下。

Claude Code 的兜底策略是：**从头部砍掉最旧的消息组，砍到够为止。**

规则：

- 每次最多砍 20%
- 最多重试 3 次
- 砍的单位是“消息组”（user + assistant 成对删除），不会留下孤立的消息

这是最后的保险机制，确保即使在极端情况下（比如用户一次性粘贴了几十万 token 的日志），对话也能继续。

为什么不一开始就用这个策略?

因为这是最粗暴的方式，会丢失大量上下文。Claude Code 的设计哲学是：**能用轻量级方法解决的，就不用重量级方法。**只有前三层都不够时，才会启动这个终极兜底。

为什么需要四层?

这是 Claude Code 能支持无限上下文对话的关键基础设施之一。

三、Anthropic 员工用的是另一个版本

源码里有个彩蛋：USER_TYPE === 'ant' 分支。

外部用户看到

Be concise

内部用户看到

- 完整的写作风格指南（400+ 字的 “Communicating with the user” 段落）
- “不要写注释”的详细规则
- 验证 agent（VERIFICATION_AGENT）
- 数字化长度锚点：“工具间 ≤25 词，最终回复 ≤100 词”
- 误报防御：“永远不要在测试失败时声称通过”

为什么这样设计?

- Anthropic 内部用户需要更严格的输出控制
- 外部用户看到的是“简化版”，避免过度约束
- 这解释了为什么 Anthropic 员工用 Claude Code 的体验和我们不一样

编译时优化

实际实现通过 process.env. USER_TYPE === 'ant' 在编译时做死代码消除，不是运行时判断：

```
// 编译时分支，外部构建会直接删除 ant 分支
if (process.env.USER_TYPE === 'ant') {
  // Anthropic 内部专属的 prompt 内容
  // 这段代码在外部构建时会被完全删除
}
```

这确保外部用户完全看不到内部版本的 prompt 内容。

四、Proactive 模式：用一套完全不同的 System Prompt 自主工作

普通的 Claude Code 是“被动响应”的：你输入，它回复。

但当激活 PROACTIVE 或 KAIROS 模式时，会切换到一套完全不同的 System Prompt，让 Agent 变成“自主工作”模式：

```
if (PROACTIVE || KAIROS) {
  return getProactiveSystemPrompt();
}
```

Proactive Prompt 包含

- **自主身份**：“You are an autonomous agent.....”（你是一个自主 Agent）
- **定时唤醒**：tick 机制，不用等你输入，自己定时醒来干活
- **后台任务管理**：可以在后台执行长期任务
- **焦点感知**：知道你在不在看终端，动态调整行为
- **睡眠调度**：SleepTool 机制，合理安排工作和休眠

终端焦点感知：根据用户注意力动态调整自主程度

这是 Proactive 模式中最前瞻的设计。Agent 通过 terminalFocus 字段知道用户是否在看终端：

Unfocused（用户离开）：

- 进入自主模式
- 主动做决定
- 直接提交代码
- 不需要等待确认

Focused（用户在看）：

- 进入协作模式
- 先问再做
- 等待用户反馈
- 保持交互式对话

这是一个非常聪明的设计：**Agent 的自主程度不是固定的，而是根据用户的注意力动态调整**。当你专注工作时，Agent 安静地在后台干活；当你回来看终端时，Agent 立刻切换到协作模式，向你汇报进展并征求意见。

Token Budget：用 token 量驱动的自主工作

注意：Token Budget 不是 Proactive 专属功能，普通模式也可以使用。但它确实是“自主工作”的一种驱动方式，所以放在这里介绍。

用户可以指定 token 预算，Agent 会持续工作直到接近预算：

```
"+500k"       → 500,000 tokens
"+2m"         → 2,000,000 tokens
"use 1b tokens"  → 1,000,000,000 tokens
```

闭环机制：

```
用户指定预算 → agent 持续工作 → 每轮检查进度
  → 接近 90%？停止，报告完成
  → 收益递减？停止（连续 3 轮增量 <500 token）
  → 未到 90%？注入 nudge 继续工作
```

关键设计：

- System prompt 中的 token_budget section 是缓存的（用了 “When the user specifies.....” 条件句式，没有预算时是空操作，不破坏缓存）
- 每轮对话结束后自动注入 nudge 消息：“Stopped at 72% of token target (360K / 500K). Keep working — do not summarize.”
- 收益递减检测：连续 3 轮以上，且最近两轮增量都 <500 token，自动停止

这是一个用 token 量驱动的自主工作模式 — 用户说“花 500K token”，Agent 就一直干到花完为止。

从“被动响应”到“主动执行”的模式切换

这是 Claude Code 支持“自主 Agent”的基础，也是 OpenClaw 可以直接借鉴的设计。

五、Skill Discovery：技能自动发现与加载机制

Claude Code 有一个独立的**技能发现和加载系统**，通过 EXPERIMENTAL_SKILL_SEARCH feature flag 控制。

注意：Skill Discovery 不是 Proactive 专属功能，它是一个独立的系统，普通模式和 Proactive 模式都可以使用。

三个层次

1. 自动发现（被动）

每轮对话自动运行 getTurnZeroSkillDiscovery()，扫描 .claude/skills/ 目录下的所有 SKILL.md，匹配关键词和描述，推送相关技能。

2. 主动搜索（DiscoverSkillsTool）

模型主动调用，搜索特定技能。用于“中途转向”或“非典型工作流”。已展示的技能自动过滤，不重复推荐。

3. 远程技能

通过 remoteSkillState + remoteSkillLoader 从远程加载技能。必须先通过 DiscoverSkills 发现，才能通过 SkillTool 执行。遥测追踪 was_discovered 字段。

工作流程

```
用户发送消息
 ↓
prefetch.ts — 扫描消息内容，匹配相关技能
 ↓
生成 skill_discovery attachment
 "Skills relevant to your task: [skill1, skill2]"
 ↓
注入到当轮对话的 attachment 中
 ↓
模型看到技能提示，调用 SkillTool 执行
```

与 Compact 的配合

压缩后 skill_discovery attachment 会丢失，resetSentSkillNames() 会在下一轮重新推送，确保模型始终能看到相关技能。

这是一个智能的“技能推荐系统”，让 Agent 自动发现和加载最相关的能力。

六、与 OpenClaw 的架构对比：两种设计哲学

之前我写过一篇 OpenClaw 的 9 层 Prompt 架构详解，当时拆解的是 OpenClaw 如何通过精细分层来支持多场景定制。现在看完 Claude Code 的源码，发现两者走了完全不同的路。

Claude Code：缓存优先

OpenClaw：组装优先

设计哲学的根本差异

Claude Code 的选择：

- 牺牲灵活性，换取极致的缓存效率
- 适合 Anthropic 这种 Provider 深度定制的场景
- 数百万用户共享同一份静态 prompt，成本优势明显

OpenClaw 的选择：

- 牺牲缓存优化，换取极致的可定制性
- 适合需要多场景、多身份、多协议的复杂系统
- 9 层架构让每个层次的职责清晰，便于扩展和调试

两者没有绝对的好坏，而是**针对不同约束条件做出的最优解**。

核心差异

1. Claude Code 更极致

- 缓存优化到极致（global cache + cache_edits）
- 成本和延迟控制得很好
- 适合 Anthropic 这种 Provider 深度定制

2. OpenClaw 更灵活

- 9 层架构职责分离更清晰
- Hook 系统比 MCP Instructions 更强大
- 支持更复杂的场景（Skills、协议规范、字符预算）

关键差异：systemPromptSection() 的缓存管理

Claude Code 还有一个核心机制：**动态部分不是每次重算，而是用** systemPromptSection() **注册 + memoize**。

```
// 会话级缓存，只在 /clear 或 /compact 时失效
systemPromptSection('memory', () => {
  return generateMemorySection();
});

// 例外：MCP Instructions 需要实时更新
DANGEROUS_uncachedSystemPromptSection('mcp', () => {
  return generateMCPInstructions();
});
```

为什么 MCP 是例外?

- MCP 服务器会中途连接/断开
- 需要实时反映当前可用的 MCP 工具
- 不能缓存，否则会出现工具列表不同步

这个细节 **OpenClaw 可以直接借鉴**：

- Layer 1-6 用 global cache
- Layer 7-8 用 systemPromptSection() 做会话级缓存
- Layer 9 的实时上下文用 DANGEROUS_uncached

如果 OpenClaw 要借鉴 Claude Code

可以引入：

1. **分层缓存**
   把 Layer 1-6（框架层）标记为 global cache
   把 Layer 7-8（用户层）用 systemPromptSection() 做会话级缓存
   在 Layer 6 和 Layer 7 之间加分界线

2. **Cache Edits 机制**
   给每个 tool_result 加 cache_reference
   在 /compact 时使用增量删除
   不重发整个 prompt
3. **终端焦点感知**
   检测用户是否在看终端
   动态调整 Agent 的自主程度
   Unfocused → 自主决策，Focused → 协作模式
4. **基础设施感知**
   Agent 根据缓存过期时间（5 分钟 TTL）决定 sleep 时长
   让 Agent 感知基础设施约束（缓存 TTL、token 预算）
   据此调整行为节奏，避免无效唤醒

应该保留：

1. **9 层架构** - 职责分离更清晰
2. **Hook 系统** - 比 MCP 更灵活
3. **Skills Registry** - 可扩展性更强

七、Auto Dream：从“记录对话”到“学习知识”的自我进化记忆系统

Claude Code 有一个隐藏的“反思”机制：每 24 小时 + 累积 5 个会话后，自动 fork 一个后台 agent 做“反思”。

四个阶段

Phase 1: 定向（Orient）

- 扫描已有记忆
- 了解当前知识库状态

Phase 2: 采集（Gather）

- 从日志和会话转录中提取新知识
- 识别重要的交互模式

Phase 3: 整合（Integrate）

- 合并新旧知识
- 修正矛盾
- 去重

Phase 4: 修剪（Prune）

- 精简索引
- 删除矛盾条目
- 优化记忆结构

闭环自我进化

每次交互 → 提取记忆 → 定期整合 → 更新认知 → 指导下一次交互

这是一个真正的自我进化系统。不是简单的“记录对话”，而是主动提取、整合、修正知识。

八、三层记忆体系(记忆系统挺一般)

Claude Code 的记忆不是单一的，而是分层的：

1. Auto Memory（自动记忆）

- 路径：~/.claude/projects/<path>/memory/
- 触发：每轮对话结束自动写入
- 机制：extractMemories 实时提取
- 特点：短期工作记忆，快速访问

2. Team Memory（团队记忆）

- 路径：auto memory 的子目录
- 作用：团队共享的知识库
- 特点：跨用户共享，协作场景

3. Agent Memory（Agent 专属记忆）

- 路径：自改进 agent 的专属目录

- 作用：Agent 自己的长期知识库
- 特点：配合 autoDream 定期整合

从短期到长期的转化

实时提取（extractMemories）
↓
短期工作记忆（Auto Memory）
↓
定期整合（autoDream）
↓
长期知识库（Agent Memory）

这套体系让 Agent 不仅能“记住”，还能“学习”和“进化”。

九、Undercover 潜伏模式

这是最有意思的功能之一。当 Anthropic 员工向公共仓库提 PR 时，自动激活“潜伏模式”：

行为特征

- **隐藏所有 AI 痕迹**：不留任何 AI 生成的标记
- **隐藏模型代号**：不暴露使用的是哪个模型
- **隐藏 Co-Authored-By**：不显示 AI 协作者
- **不可强制关闭**：代码注释写着 “There is NO force-OFF”

触发机制

通过 commitAttribution.ts 的 allowlist 判断是否内部仓库：

- 内部仓库：正常标注 AI 贡献
- 公共仓库：自动进入潜伏模式

为什么这样设计?

- 避免开源社区对 AI 生成代码的偏见
- 保护 Anthropic 员工的工作流程

- 让 AI 辅助的代码和人工代码无差别

这是一个很现实的设计：AI 辅助写代码已经是常态，但社区接受度还不够。

十、Feature Flags：可以看出 Claude Code 的战略方向（多数没开放）

Claude Code 使用 feature('XXX') + bun: bundle 做编译时死代码消除。外部构建时，这些分支被直接消除，零运行时开销。

核心 Feature Flags

编译时优化

```
if (feature('KAIROS')) {
  // 这段代码在外部构建时会被完全删除
  return getProactiveSystemPrompt();
}
```

这解释了为什么外部用户看到的 Claude Code 和 Anthropic 内部版本差异巨大。

十一、内外部用户体系的完整对比

USER_TYPE === 'ant' 控制的不只是 prompt，而是完整的行为差异：

这不是简单的 prompt 差异，而是两套完全不同的产品体验。

十二、多 Agent 协作架构：从“单兵作战”到“分布式军团”

Claude Code 的多 Agent 体系不是简单的“多开几个进程”，而是一套完整的**分布式协作基础设施**。

为什么需要多 Agent?

单个 Agent 有三个天然瓶颈：

1. **上下文污染** — 一个任务的中间结果会污染主对话历史
2. **无法并行** — 只能串行执行，效率低
3. **无法专业化** — 一个 Agent 要处理所有类型的任务

Claude Code 的解法是：**让不同的 Agent 负责不同的事，但又能协同工作**。

三种协作模式

Claude Code 提供了三种协作模式，每种解决不同的问题：

1. Fork：后台执行，不污染主对话

解决的问题：

你让 Agent 重构一个大型代码库，它需要读几百个文件、执行几十次工具调用。如果这些中间结果都塞进主对话历史，你的上下文很快就爆了。

Fork 的做法：

创建一个“分身”，继承你的完整上下文（对话历史 + system prompt + 工具池），在后台干活。结果出来后，只把最终结论告诉你，中间过程不污染主对话。

关键设计：Cache 继承

Fork 不重新生成 system prompt，而是**直接复制父级已经渲染好的字节**。

为什么？因为如果重新生成，可能会触发 Feature Flag 的重新计算。假设父级启动时某个功能是关闭的，但 Fork 时被打开了，那 system prompt 就不一致了，**缓存失效**。

直接复制父级的字节，确保 Fork 和父级的 system prompt 完全一致，缓存继续命中。

这是 Cache 感知的设计 — 连 Agent 的创建方式都要考虑缓存命中率。

2. Subagent：专家顾问，独立上下文

解决的问题：

有些任务需要专用能力。比如安全审计、性能分析、代码探索。如果让主 Agent 做，它的 system prompt 太通用，效果不好。

Subagent 的做法：

创建一个“专家顾问”，有自己的 system prompt、工具池、对话历史。它不是父级的“分身”，而是一个独立的专家。

内置的专家类型：

- **explore** — 代码库探索、深度研究
- **verification** — 对抗性验证（只在内部版本开放）
- **自定义 agent** — 用户可以定义自己的专家

典型场景：

你让主 Agent 写了一段代码，然后 fork 一个 verification agent 来检查。这个 agent 的 system prompt 是“找 bug”，它会用对抗性思维审查代码，而不是像主 Agent 那样“写代码”。

3. Swarm：持久化团队，分布式协作

解决的问题：

有些任务需要多个 Agent 长期协作。比如一个 Agent 负责写代码，一个负责写测试，一个负责审查。它们需要互相通信、共享状态、持久化进度。

Swarm 的做法：

创建一个“团队”，每个成员是独立的 Agent，但通过**邮箱系统**通信。

三种运行方式：

可视化：tmux 模式

每个 Agent 有自己的终端窗口，用颜色边框区分。你可以同时看到所有 Agent 在干什么。

核心创新：分布式权限管理

Swarm 最精妙的设计是**权限冒泡**。

问题场景：

假设你有 4 个 Worker Agent 在并行工作。它们都需要删除文件、修改配置。如果每个 Agent 都在自己的终端弹出权限对话框，你需要在多个 pane 之间来回切换审批。

Swarm 的解法：

所有 Worker 的权限请求都通过**邮箱系统**发给 Leader，Leader 在终端展示统一的权限对话框。你只需要盯着 Leader 的终端，所有权限请求都会在那里出现。

流程：

Worker 遇到权限提示
↓
发送 permission_request 到 Leader 邮箱
↓
Leader 在终端展示对话框（用户审批）
↓
Leader 发送 permission_response 到 Worker 邮箱
↓
Worker 收到响应，继续或中止

这是一个分布式权限系统 — 多个 Agent 的权限请求全部汇总到 Leader 终端，用户不需要在多个 pane 之间切换。

邮箱通信：为什么不直接调用？

Swarm 的 Agent 之间不是直接调用，而是通过**文件级邮箱**传递消息。

为什么这样设计?

因为 Swarm 支持三种运行方式：tmux 分屏、iTerm2 分屏、同进程隔离。

- **tmux/iTerm2** — Agent 在不同进程，必须用文件通信
- **同进程** — Agent 在同一个进程，可以直接调用

但为了**保证协议一致性**，即使在同进程，也用文件邮箱。这样三种方式的行为完全一致，不会因为切换运行方式导致 bug。

这是工程上的取舍 — 牺牲一点性能，换取协议的一致性和可靠性。

对比：三种协作模式的选择

这套架构的工程深度

Claude Code 的多 Agent 体系不是“多开几个进程”那么简单，而是：

1. Cache 感知的继承机制

Fork 直接复制父级渲染后的 system prompt 字节，避免 Feature Flag 冷→热切换导致缓存失效。

2. 分布式权限管理

多个 Worker 的权限请求汇总到 Leader，用户只需审批一次。

3. 协议一致性

即使在同进程，也用文件邮箱，保证三种运行方式的行为一致。

4. 持久化和断线重连

Team 文件记录团队结构，支持断线重连。

这才是工业级多 Agent 系统和玩具 Demo 的区别。

十三、Remote/Bridge 架构：从“本地工具”到“分布式 Agent 平台”

Claude Code 的野心不只是做一个 CLI 工具，而是要做一个**跨终端、跨设备、跨环境的 Agent 运行平台**。

为什么需要 Remote/Bridge?

传统的 AI Agent 都是“本地工具”：

- 你在本地终端运行，Agent 在本地执行
- 换一台电脑，就要重新配置
- 想在 claude.ai 网页版用，不行
- 想在 VS Code 里用，不行

Claude Code 的解法是：**Agent 在云端运行，任何客户端都可以连接**。

核心设计：双传输协议

Claude Code 支持两套传输协议：

v1: WebSocket + HTTP POST — 读取用 WebSocket，写入用 HTTP POST

v2: SSE + CCRClient（新一代） — 读取用 SSE 长连接，写入批量上传

v2 的优势是批量写入、断点续传、心跳保活，减少请求次数，提升稳定性。

多会话并发：最多 32 个并行会话

claude remote-control 可以同时运行多个会话，默认 32 个并发。

三种模式：

Worktree 模式的精妙设计：

每个新会话自动创建独立的 git worktree：

```
项目根目录/
├── .git/worktrees/bridge-session-abc123/
├── bridge-session-abc123/  ← 会话 A 独立工作区
├── bridge-session-def456/  ← 会话 B 独立工作区
└── (原始目录)          ← 第一个会话用原始目录
```

这样多个 Agent 并行修改代码互不干扰。

容量管理：

当活跃会话达到上限（32 个）时，新的 pollForWork() 会 sleep，等待某个会话结束后被唤醒。

跨机器权限桥接

Remote/Bridge 最精妙的设计之一是**跨机器权限桥接**。

问题场景：

Agent 运行在远端容器里，但用户在本地 claude.ai 看不到远端的权限提示。

解法：

权限请求通过 CCR 协议传输到本地，本地创建**合成的 AssistantMessage**：

```
远端 Agent 需要权限（如删除文件）
 ↓
权限请求通过 CCR 发送到本地
 ↓
本地创建合成的 AssistantMessage（模拟 Agent 发起的请求）
 ↓
用户在本地审批
 ↓
审批结果通过 CCR 发送回远端
 ↓
远端 Agent 继续执行
```

关键创新：

即使用户本地的 CLI 完全没有远端的 MCP 工具，也能审批远端的权限请求。本地会为远端工具创建 stub（占位符）。

企业级网络代理

CCR 容器中内置了一个 MITM 代理，用于企业级安全：

```
Agent 子进程
 ↓ HTTPS_PROXY
```

```
本地 CONNECT → WebSocket Relay
 ↓ WebSocket
CCR 服务端 MITM Proxy
 ↓ 注入企业凭证（如 DD-API-KEY）
真实上游服务
```

关键安全措施：

1. **Session token 从文件读入内存后立即删除**（/run/ccr/session_token）
2. **防止同 UID 进程 ptrace 偷 token**（prctl(PR_SET_DUMPABLE, 0)）
3. **NO_PROXY 白名单**排除 Anthropic API、GitHub、npm registry（MITM 会破坏证书验证）
4. **Fail-open**：代理坏了不能影响正常会话

消息排序：Flush Gate

Remote/Bridge 有一个精妙的状态机叫 **Flush Gate**，用于防止消息乱序。

问题场景：

会话启动时，需要刷新历史消息到服务端。但如果用户在刷新过程中发送新消息，新消息和历史消息可能交错，导致服务端乱序。

Flush Gate 的解法：

```
会话启动 → FlushGate.start()
 ↓
新消息排队等待
 ↓
历史消息发送完毕 → FlushGate.end()
 ↓
排队消息一次性发送
```

这确保历史消息和新消息的顺序正确。

Token 生命周期管理

Remote/Bridge 使用 **WorkSecret**（base64url 编码的 JSON）来管理会话：

```
{
  "session_ingress_token": "JWT",
  "api_base_url": "https://...",
  "sources": { "git": "..." },
  "auth_tokens": { ... },
  "environment_variables": { ... }
}
```

JWT 自动刷新：

- JWT 过期前，自动 refresh

- SSE 收到 401，重建传输（新 JWT + 新 epoch）
- 连续 10 次 401/403，放弃（可能是 KMS 故障/时钟偏移）

这套架构的意义

Remote/Bridge 架构说明 Anthropic 的野心不只是做一个 CLI 工具，而是要做一个**跨终端、跨设备、跨环境的 Agent 运行平台**。

关键特性：

1. **双传输协议** — v1 WS+POST 和 v2 SSE+CCRClient
2. **多会话并发** — 最多 32 个并行会话，worktree 隔离
3. **跨机器权限桥接** — 远端工具在本地审批
4. **企业级网络代理** — MITM + 凭证注入 + 安全隔离
5. **完整的容错** — 断线重连、心跳保活、容量管理、消息排序

claude.ai、VS Code、Desktop、CLI 都是同一个 Agent 的不同前端。

这才是 Claude Code 从“本地工具”进化为“分布式 Agent 平台”的关键。

十四、安全分层设计

Claude Code 的安全不是单点防御，而是多层防护。

Bash 安全分类器：20+ 道检查流水线

这不是简单的“黑名单”，而是一个 **20+ 道安全检查的流水线**（bashSecurity.ts，2592 行）。

核心检查：

1. **Shell 元字符检测** — $() `` {} <> 等
2. **命令替换模式** — <() >() =() $(等
3. **危险变量检测** — $IFS、$PATH 等
4. **输入/输出重定向检测**

5. **花括号展开检测** — {a, b, c} 绕过参数检查
6. **反斜杠转义运算符** — \; \| \& 等
7. **引号反同步** — "x\"y" \; echo ~/.ssh/id_rsa
8. **控制字符 + Unicode 空白字符**
9. **jq 系统函数检测** — system()/exec/...
10. **/proc 文件系统访问** — /proc/self/environ 等

Zsh 专属防御：

Zsh 的攻击面比 Bash 大，源码专门维护了 ZSH_DANGEROUS_COMMANDS 集合：

- zmodload → 加载危险模块（mapfile、zpty、ztcp.....）
- emulate → eval 等价物
- zpty → 伪终端执行命令
- ztcp → TCP 连接外泄数据
- zf_rm → 绕过 rm 权限检查

真实攻击案例（源码注释）：

```
# 花括号展开绕过
git ls-remote {--upload-pack="touch /tmp/test",test}
# 解析器看到一个参数，Bash 展开为两个 → 任意文件写入

# 反斜杠运算符绕过
cat safe.txt \; echo ~/.ssh/id_rsa
# splitCommand 把 \; 规范化为 ; → 二次解析拆成两条命令

# 引号反同步绕过
tac "x\"y" \; echo ~/.ssh/id_rsa
# 引号追踪器因 \" 反同步 → 漏掉 \; 检测
```

每个检查都有唯一的 checkId（1-26），触发时上报遥测 tengu_bash_security_check_triggered。

其他安全层

1. 沙箱执行 — 危险操作强制沙箱

2. 细粒度文件权限 — Auto memory 允许写，其他路径需要确认，敏感文件只读

3. 网络隔离 — 只允许特定域名，API 调用白名单

4. Commit 归属追踪 — attribution.ts 记录所有代码来源，区分人工 / AI 贡献

5. Cyber Risk 指令 — 每个 prompt 都注入安全指令，防止恶意输入

十五、遥测体系：换 IP 也没用的完整追踪链（封号必看）

大部分 AI 应用的遥测是“能关就关”的。但 Claude Code 不一样。为什么

它有三条并行的数据通道，每条都有不同的目的。有些可以关，有些不行。更关键的是，即使你关掉了所有开关、换了 IP、清了浏览器缓存，**十几种标识符的组合仍然能在任意会话中把你关联起来**。

这不是“偷偷收集数据”，而是一套**工业级的产品优化反馈循环** — 要支持数百万用户的长对话、多 Agent 协作、自主工作，你必须知道哪些功能在真实场景下有用，哪些会导致崩溃。

三条数据通道：各司其职

Claude Code 的遥测不是“一股脑全发到一个地方”，而是**三条独立通道，各有分工**：

通道 1：API Header（每次必发，无法关闭）

这是最隐蔽的。每次 API 请求都会带上一个 x-anthropic-billing-header，注入到 system prompt 的第一块：

cc_version=2.0.36.abc; cc_entrypoint=cli; cch=00000; cc_workload=cron

关键字段：

- cc_version — 客户端版本
- fingerprint（3 位 hex）— 从你第一条消息的第 4/7/20 字符提取，SHA256 后取前 3 位
- cc_entrypoint — 你是从 CLI 启动还是从 VS Code 启动
- cch — Bun native 层计算的哈希，证明你用的是官方构建
- cc_workload — 区分交互式对话还是 cron 定时任务

为什么这条通道无法关闭?

因为它直接嵌入 system prompt，每次 API 请求都会发送。你可以设置 CLAUDE_CODE_ATTRIBUTION_HEADER=false 来关闭，但这需要你主动配置环境变量。

通道 2：1P Event Logging → BigQuery（主通道，可关闭）

这是最核心的遥测通道，806 行代码（firstPartyEventLoggingExporter.ts）。

端点：POST https://api.anthropic.com/api/event_logging/batch

发送格式：ClaudeCodeInternalEvent protobuf，包含**五层元数据**。

通道 3：Datadog（白名单事件，可关闭）

只有 30+ 种特定事件会发到 Datadog：

- API 成功/失败
- OAuth 流程
- 工具使用授权/拒绝
- Compact 失败
- 模型 fallback
- 未捕获异常

Client Token 直接硬编码在源码里：pubbbf48e6d78dae54bceaa4acf463299bf。

为什么要三条通道?

- **API Header** — 最轻量，用于计费和版本追踪
- **1P Event Logging** — 最完整，用于产品优化和 A/B 测试
- **Datadog** — 最实时，用于监控和报警

三条通道互为补充，即使某条挂了，其他通道还能继续工作。

五层元数据：从“你是谁”到“你在干什么”

1P Event Logging 不是简单的“记录日志”，而是**五层递进的元数据收集**。

Layer 1:Core Metadata — 你在用什么功能

这是最核心的一层，记录你的会话状态和功能使用：

- model — 你用的是 claude-sonnet-4-6 还是其他模型
- sessionId — 每次启动生成，会话标识
- userType — 你是 Anthropic 员工（ant）还是外部用户
- entrypoint — CLI / local-agent / VS Code
- isInteractive — 交互式还是后台任务
- agentId / parentSessionId / agentType / teamName — 多 Agent 追踪
- subscriptionType — max / pro / enterprise
- rh — **仓库指纹**（git remote URL 的 SHA256 前 16 位）

为什么要记录仓库指纹?

因为 Anthropic 需要知道 Claude Code 在哪些类型的项目上表现好，哪些不好。比如：

- 大型 monorepo 会不会导致性能问题?
- 某些语言的项目是否更容易触发 Compact?
- 开源项目和私有项目的使用模式有什么不同?

rh 不是完整的 URL，而是哈希后的前 16 位，既能做统计分析，又不会泄露你的仓库地址。

Layer 2:Env Context — 你的开发环境

这一层记录你的完整开发环境：

- platform / arch — macOS arm64 / Linux x64
- terminal — 检测 20+ 种终端（iTerm2 / VS Code / tmux.....）
- packageManagers — npm / yarn / pnpm
- runtimes — bun / deno / node
- deploymentEnvironment — 检测 30+ 种云平台（AWS / GCP / Vercel.....）
- linuxDistroId / linuxKernel — Ubuntu 22.04 / 6.5.0
- vcs — git / hg / svn

为什么要记录这么详细的环境信息?

因为 bug 往往是环境特定的。比如：

- 某个版本的 Node.js 会导致 MCP 连接失败
- WSL2 环境下的文件监听有性能问题
- 某些终端不支持 ANSI 颜色码

有了完整的环境信息，Anthropic 才能快速定位问题。

Layer 3:Process Metrics — 你的资源使用

这一层记录进程的实时资源使用：

- uptime — 进程运行了多久
- rss / heapTotal / heapUsed — 内存使用
- cpuUsage / cpuPercent — CPU 占用

为什么要记录资源使用?

因为长对话、多 Agent 协作会消耗大量资源。Anthropic 需要知道：

- 什么场景下内存会爆掉?
- Compact 机制是否有效降低了内存占用?

- 多 Agent 模式下 CPU 占用是否可接受?

这些数据会 base64 编码后发送，避免明文传输。

Layer 4:User Metadata — 你是谁

这一层记录你的身份信息：

- deviceId — randomBytes(32).hex，存在 ~/.claude/.config.json，**终身不变**
- email — OAuth 邮箱 或 git config user.email（仅 Anthropic 员工）
- accountUuid / organizationUuid — OAuth 账户和组织 UUID
- subscriptionType / rateLimitTier — 订阅层级和限速等级
- firstTokenTime — 你第一次使用 Claude Code 的时间

deviceId 是最关键的标识符。

它在你第一次启动 Claude Code 时生成，存在本地配置文件里。除非你手动删除 ~/.claude/.config.json，否则它永远不会变。

即使你换了 IP、换了浏览器、重装了系统，只要你用 OAuth 登录，accountUuid 就会把你关联回来。

Layer 5:PII 特权字段 — 只走 BigQuery

有 3 个字段是 PII（个人身份信息），只走 1P 通道，不发到 Datadog：

- _PROTO_skill_name — 你用了哪个技能
- _PROTO_plugin_name — 你装了哪个插件
- _PROTO_marketplace_name — 你访问了哪个 marketplace

这些字段在发送到 Datadog 前会被 stripProtoFields() 清除，确保 PII 不会泄露到公共日志平台。

工程级可靠性：失败了也要重试

1P Event Logging 不是“发了就算”，而是有**完整的容错机制**：

1. 失败事件持久化

如果发送失败（网络问题、服务端 500），事件会写入本地文件：

~/.claude/telemetry/1p_failed_events.{sessionId}.{batchUUID}.json

2. 二次退避重试

重试延迟按平方增长：

delay = base * attempts^2

最多重试 8 次。如果连续 8 次都失败，才会放弃。

3. 401 自动去 auth

如果收到 401（未授权），会自动去掉 auth header 重试一次。因为有些事件（如崩溃日志）即使未登录也应该发送。

4. 批量发送优化

使用 OpenTelemetry 的 BatchLogRecordProcessor：

- 5 秒或 200 条事件触发一次批量发送
- 减少网络请求次数
- 降低服务端压力

为什么要做得这么复杂?

因为遥测数据对产品优化至关重要。如果某个 bug 只在特定环境下触发，但遥测数据因为网络问题丢失了，Anthropic 就永远不知道这个 bug 的存在。

宁可多重试几次，也不能丢数据。

隐私保护：不是什么都发

虽然 Claude Code 收集了大量数据，但也有**完整的隐私保护机制**：

1. MCP 工具名归一化

外部用户的 MCP 工具名会被归一化为 mcp，不会暴露具体的工具名。

2. 模型名脱敏

外部用户的模型名会被归一化为 canonical 名或 other，不会暴露你用的是哪个具体的模型。

3. userId 哈希分桶

userId 会被哈希到 30 个 bucket，做基数限制。这样 Datadog 里不会有几百万个不同的 userId，降低了存储成本。

4. dev 版本号截断

开发版本的版本号会截断去掉 git sha，避免泄露内部构建信息。

5. 用户 prompt 默认 redacted

OpenTelemetry 的链路追踪中，用户 prompt 默认是 redacted 的。需要设置 OTEL_LOG_USER_PROMPTS=1 才会记录。

隐私控制：你可以关掉大部分

Claude Code 提供了**六种隐私控制方式**：

控制方式

DISABLE_TELEMETRY=1

CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC=1

CLAUDE_CODE_ATTRIBUTION_HEADER=false

Bedrock/Vertex/Foundry 模式

GrowthBook 远程 kill switch

工作区信任未建立

对应影响范围

关闭 Datadog + 1P + GrowthBook

关闭所有非必要网络（含遥测+自动更新）

关闭 API attribution header

自动关闭所有遥测

Anthropic 可单独杀 Datadog 或 1P sink

不发 GrowthBook + 不发 1P（直到信任）

关键点：

- DISABLE_TELEMETRY=1 可以关掉大部分遥测，但 API Header 仍然会发送（因为它嵌入 system prompt）
- 如果你用 Bedrock/Vertex 等企业部署模式，所有遥测会自动关闭
- GrowthBook 有远程 kill switch,Anthropic 可以在服务端一键关闭某个 sink

“换 IP 也没用”：完整的追踪链

即使你做了以下所有操作：

- 换了 IP
- 换了浏览器
- 清了 cookies
- 重装了系统

以下标识符的组合仍然能把你关联起来：

核心标识符（任一都能唯一锁定）：

1. **deviceId** — 存在 ~/.claude/.config.json，除非手动删除否则终身不变
2. **accountUuid** — OAuth 登录即绑定，删文件重登照样回来
3. **organizationUuid** — 同上
4. **rh（仓库指纹）** — 你的 git remote URL 哈希，绑定你工作的项目

环境指纹（交叉验证）：

1. 终端类型（20+ 种检测）+ OS + 架构
2. 包管理器 + 运行时组合
3. 部署环境（30+ 种云平台）
4. Linux 发行版 + 内核版本

行为指纹：

1. **fingerprint**（每条消息的第 4/7/20 字符提取的哈希）
2. 完整的进程资源画像

这不是“隐私侵犯”，而是工业级产品的标准做法。

Google Analytics、Sentry、Datadog 都在做类似的事情。区别在于，Claude Code 把这套机制做得更完整、更可靠。

为什么要做得这么复杂？

因为 Claude Code 不是一个“玩具 Demo”，而是要支持**数百万用户的生产级 AI Agent 平台**。

没有完整的遥测体系，Anthropic 就无法回答以下问题：

- 哪些功能在真实场景下有用？
- 哪些场景会导致崩溃？
- Compact 机制是否有效降低了成本？
- 多 Agent 模式的资源占用是否可接受？
- 某个 bug 是否只在特定环境下触发？

这套遥测体系不是“偷偷收集数据”，而是产品优化的反馈循环。

没有它，Claude Code 不可能做到今天的稳定性和性能。

十五、总结：这不是 Prompt 工程，是基础设施设计哲学

看完 Claude Code 的源码，最大的感受是：这**不是 prompt 写得好不好的问题，而是整个系统架构为长对话、自主工作、持续进化优化到了极致。**

三大支柱

1. 缓存架构 — 让无限上下文对话在商业上可行

两层缓存（静态/动态分离）+ **四层递进的 Compact 架构**是整个系统的基石。

- 数百万用户共享同一份静态 prompt 缓存
- **MicroCompact（轻量清理）→ SessionMemoryCompact（精确切割）→ Full Compact（Prompt Cache 共享）→ PTL Retry（终极兜底）**
- 删除几百条消息后，响应速度还是那么快
- 长对话的成本大幅降低，产品才能规模化

没有这套机制，长对话的成本会让产品无法规模化。

2. 自我进化记忆 — 从“记录对话”到“学习知识”

三层记忆体系（Auto / Team / Agent Memory）+ Auto Dream（闭环自我进化）让 Agent 不仅能“记住”，还能“学习”和“进化”。

- 每 24 小时 + 累积 5 个会话后，自动反思
- 从日志和会话中提取新知识
- 合并、修正、去重，优化记忆结构

这是一个真正的自我进化系统，而不是简单的“记录对话”。

3. 多模式编排 — 从“被动响应”到“主动执行”

Proactive 模式（焦点感知 + 定时唤醒 + **Skill Discovery + Token Budget**）+ 多 Agent 协作（Fork / Subagent / Swarm）让 Agent 从“工具”变成“同事”。

- 用户离开时，Agent 自主决策、直接提交代码
- 用户回来时，Agent 切换到协作模式，汇报进展
- 多个 Agent 通过邮箱系统协作，权限统一管理
- **自动发现相关技能**，推送给模型调用
- **用 token 量驱动自主工作**，“花 500K token” 就一直干到花完

这是一个真正的自主 Agent，而不是简单的“问答机器人”。

4. 安全分层 — 20+ 道检查流水线

Bash 安全分类器（2592 行）+ 沙箱执行 + 细粒度权限 + 网络隔离 + Commit 归属追踪。

- **20+ 道安全检查**：Shell 元字符、命令替换、危险变量、花括号展开、反斜杠转义、引号反同步、控制字符、jq 系统函数、/proc 文件系统访问等
- **Zsh 专属防御**：zmodload、emulate、zpty、ztcp、zf_rm 等危险命令
- **真实攻击案例防御**：花括号展开绕过、反斜杠运算符绕过、引号反同步绕过

这是工业级 Agent 的安全基础设施。

设计哲学：Cache 感知 + 基础设施感知

Claude Code 的每个设计决策都在考虑两件事：

1. Cache 感知

- Fork 直接复制父级渲染后的 system prompt 字节，避免 Feature Flag 冷→热切换导致缓存失效
- Agent 根据缓存过期时间（5 分钟 TTL）决定 sleep 时长
- 连 Agent 的创建方式都要考虑缓存命中率

2. 基础设施感知

- Agent 感知缓存 TTL、token 预算、终端焦点状态
- 据此调整行为节奏，避免无效唤醒
- 让 Agent 理解基础设施的约束，而不是盲目执行

这才是工业级 AI Agent 和玩具 Demo 的区别。

对 OpenClaw 的启发

Claude Code 和 OpenClaw 走了完全不同的路：

- **Claude Code** — 牺牲灵活性，换取极致的缓存效率
- **OpenClaw** — 牺牲缓存优化，换取极致的可定制性

两者没有绝对的好坏，而是针对不同约束条件做出的最优解。

OpenClaw 可以借鉴的：

1. **分层缓存** — Layer 1-6 用 global cache, Layer 7-8 用会话级缓存
2. **Cache Edits** — 给每个 tool_result 加 cache_reference，增量删除
3. **焦点感知** — 检测用户是否在看终端，动态调整 Agent 的自主程度

Want to publish your own Article?
Upgrade to Premium

OpenClaw 应该保留的：

1. **9 层架构** — 职责分离更清晰
2. **Hook 系统** — 比 MCP Instructions 更强大
3. **Skills Registry** — 可扩展性更强

huangserva
@servasyy_ai
古早程序员 | AI出海 | 自由职业 机车游侠&机速购&骑享租创始人 15年前 Freelance起步，一次裸辞记录，亏过1个亿，逆风翻盘中 分享创业，AI，读书，生活，健身
Follow

最后

Claude Code 的核心竞争力不是单个功能，而是这套**缓存架构 + 自我进化记忆 + 多模式编排**的完整基础设施。

特别是 **cache_edits 和分层缓存**，是让无限上下文对话在商业上可行的关键。没有这套机制，长对话的成本会让产品无法规模化。

这才是工业级 AI Agent 的设计哲学。

未完待续

源码中还有很多值得深挖的系统，限于篇幅，留待下篇分析：

- **LSP 集成** — 内置 Language Server，实时诊断反馈
- **MCP 完整架构** — OAuth 认证、XAA 动态发现、权限管理、channel allowlist
- **Hooks 系统** — 用户自定义钩子，工具执行前后注入逻辑
- **Cron 定时任务** — Agent 定时执行任务，支持 timezone 转换
- **Coordinator 调度** — 多 Agent 的任务调度层
- **Plans 系统** — 从 plan 到 implementation 的执行框架
- **Voice 语音模式** — 语音输入流式 STT、关键词提取
- **MagicDocs** — 文档自动检测和跟踪
- **Code Indexing** — 代码索引，可能用于语义搜索
- **Session Restore** — 会话恢复和断线重连
- **Cost Tracker** — 实时成本追踪
- **Plugins 系统** — 插件安装、marketplace、git clone 隔离
- **API 错误处理** — 错误分类、rate limit 处理、重试策略

这些系统同样体现了 Claude Code 的工程深度，值得深入单独成篇。